

Backend Developer Case Study

Valizu Backend API

Technical Evaluation Document

Candidate: Okhtay Sattari

Position: Backend Developer

Project: Valizu Backend Case Study

API Version: 1.0.0

OpenAPI Version: 3.0.3

1. Executive Summary

This project implements the backend API for the Valizu case study.

The primary objective was to implement a reliable REST API for managing:

- Users
- Trips
- Bags
- Items
- Items contained in bags

In addition to the basic CRUD-oriented requirements, particular attention was given to the most important consistency requirement of the case study: **copying a trip safely and idempotently under concurrent requests**.

The implementation uses PostgreSQL, Prisma, TypeScript, Express.js, Zod, OpenAPI, and Vitest.

The Copy Trip operation is implemented using:

- database-level uniqueness
- SHA-256 request hashing
- transactional resource creation
- stored idempotency responses
- explicit handling of Prisma unique-constraint errors

- automated concurrency tests

The central design principle is that **correctness under concurrency must be guaranteed by the database rather than by an application-level check alone.**

2. Technology Stack

Technology	Purpose
Node.js	Runtime
TypeScript	Application language
Express.js	HTTP API framework
PostgreSQL	Relational database
Prisma	ORM and database access
Zod	Request validation
OpenAPI 3.0.3	API contract
Swagger UI	API documentation
Vitest	Automated testing
pnpm	Package management
Docker / Docker Compose	Containerized execution

3. Architecture

The application follows a modular backend structure.

The main separation is between:

1. HTTP/API layer
2. Validation layer
3. Business/service layer
4. Persistence layer
5. Database

Conceptually:

HTTP Request



Express Route



Validation / Authentication



Controller



Service



Prisma



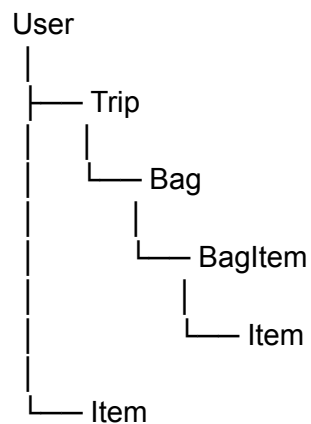
PostgreSQL

The service layer contains the business rules.

This is particularly important for Copy Trip because idempotency, transaction boundaries, ownership checks, and resource copying should not be implemented directly inside HTTP controllers.

4. Domain Model

The domain is modeled using the following relationships:



A user owns:

- Trips
- Items

A trip owns:

- Bags

A bag contains:

- BagItems

A BagItem connects:

- one Bag
- one Item

and additionally stores:

- quantity
-

5. Database Design

5.1 User

The User entity represents an API user.

Important constraints:

id UUID PRIMARY KEY
email UNIQUE

The email address is unique at the database level.

5.2 Trip

A Trip belongs to one User.

Important fields:

id
userId
name
destination
startDate
endDate
createdAt
updatedAt

The database contains an index on:

userId

This supports efficient retrieval of a user's trips.

5.3 Bag

A Bag belongs to a Trip.

Important fields:

id
tripId
name
createdAt
updatedAt

The database contains an index on:

tripId

5.4 Item

An Item belongs to a User.

Important fields:

id
userId
name
category
createdAt
updatedAt

The database contains an index on:

userId

5.5 BagItem

BagItem represents the many-to-many relationship between Bags and Items.

The model contains:

bagId
itemId
quantity

The database enforces:

UNIQUE(bagId, itemId)

This guarantees that the same item cannot appear as two independent rows in the same bag.

Quantity is stored on the relationship itself because quantity belongs to the relationship between the bag and the item.

6. Ownership and Authorization

The API uses a simplified authentication mechanism appropriate for the case study.

The authenticated user is identified through:

X-User-Id

For example:

X-User-Id: 11111111-1111-4111-8111-111111111111

The important design decision is that user ownership is not treated as a controller-only concern.

Database queries are scoped by the authenticated user wherever appropriate.

For example, Copy Trip retrieves the source trip using:

id = sourceTripId
AND
userId = authenticatedUserId

Therefore, knowing another user's UUID is insufficient to access that user's trip.

This prevents a common authorization vulnerability in REST APIs: **object-level authorization failure / IDOR-style access**.

7. API Endpoints

The API exposes the following main operations.

Health

GET /health

Returns the API health status.

Users

GET /users

Returns the users available in the case-study dataset.

Trips

GET /trips

Lists trips belonging to the authenticated user.

POST /trips/{tripId}/copy

Copies a trip, including its bags and bag items.

Bags

GET /trips/{tripId}/bags

POST /trips/{tripId}/bags

PATCH /bags/{bagId}

These endpoints provide bag listing, creation, and modification.

Bag Items

GET /bags/{bagId}/items

POST /bags/{bagId}/items

DELETE /bags/{bagId}/items/{itemId}

The Add Item operation increments quantity if the item is already present.

Items

GET /items

Lists items owned by the authenticated user.

8. OpenAPI

The API is documented using OpenAPI 3.0.3.

The specification defines:

- API metadata
- endpoints
- path parameters
- request headers
- request bodies
- authentication
- validation requirements
- response schemas
- error schemas
- HTTP status codes

The OpenAPI source is:

src/openapi.ts

The specification also documents the **Idempotency-Key** header explicitly.

9. Copy Trip Requirement

The Copy Trip operation is the most significant part of the implementation.

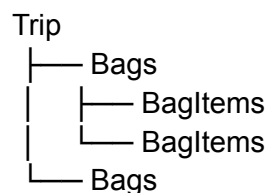
Endpoint:

POST /trips/{tripId}/copy

Required header:

Idempotency-Key: <key>

The operation copies:



The copied trip receives a new UUID.

The copied trip name is:

Copy of <source trip name>

Other trip properties are copied from the source.

10. Idempotency Design

Idempotency means that repeating the same logical request does not produce multiple side effects.

For example:

Request 1

POST /trips/123/copy

Idempotency-Key: abc-123

creates one copied trip.

If the client retries:

Request 2

POST /trips/123/copy
Idempotency-Key: abc-123

the API returns the original result.

It does not create another trip.

11. Why Application-Level Idempotency Checks Are Insufficient

A naïve implementation could be:

SELECT idempotency record

if record does not exist:

 INSERT idempotency record
 copy trip

This is vulnerable to a race condition.

Consider two concurrent requests:

Request A	Request B
SELECT	SELECT
↓	↓
not found	not found
↓	↓
INSERT	INSERT

Both requests can observe the absence of a record before either has inserted it.

The result could be two copied trips.

Therefore, the application cannot rely solely on:

check → then create

12. Database-Level Concurrency Guarantee

The Prisma schema defines:

```
@@unique([userId, operation, key])
```

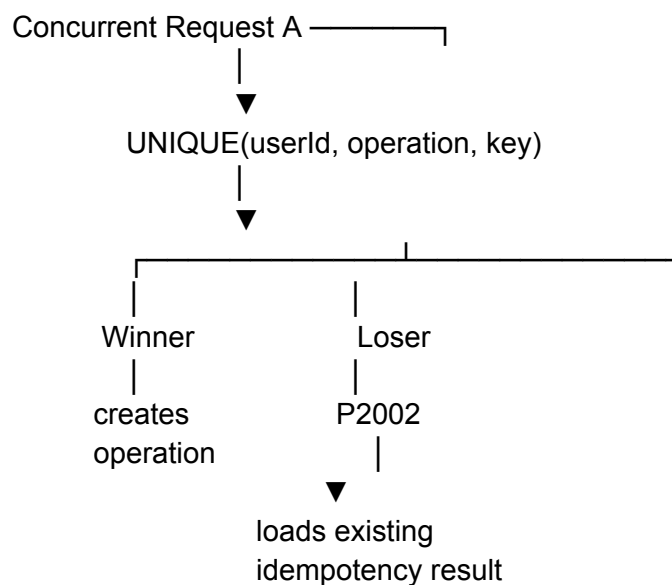
for the IdempotencyKey model.

This creates a database-level uniqueness constraint equivalent to:

```
UNIQUE(userId, operation, key)
```

The database therefore becomes the authoritative concurrency boundary.

Conceptually:



This is significantly safer than trying to coordinate concurrent requests using only JavaScript memory or application-level checks.

13. Transaction Design

The idempotency record and copied resources are deliberately created inside the same database transaction.

The transaction performs:

1. Create IdempotencyKey
2. Load source Trip
3. Create copied Trip
4. Create copied Bags
5. Create copied BagItems
6. Store responseBody
7. Store resourceId
8. Commit

If any operation fails:

ROLLBACK

Everything created inside the transaction is rolled back.

This includes the idempotency record.

14. Failure Semantics

Consider:

POST /trips/{non-existent-trip}/copy
Idempotency-Key: abc-123

The source trip lookup fails.

The service throws:

404 TRIP_NOT_FOUND

Because the operation is inside a transaction, the idempotency record is also rolled back.

Therefore:

Idempotency-Key abc-123

has not been consumed by the failed request.

The client can retry the same operation later.

This is an intentional design decision.

15. Handling Concurrent Requests

The service catches:

`Prisma.PrismaClientKnownRequestError`

and specifically checks for:

`P2002`

which represents a unique-constraint violation.

The implementation does not treat every database error as an idempotency conflict.

Only the expected unique-constraint error enters the idempotency recovery path.

This distinction is important because unrelated database errors must not be incorrectly converted into successful idempotent responses.

16. Request Hash

The service calculates a SHA-256 hash from:

`COPY_TRIP:<sourceTripId>`

The hash is stored in the `IdempotencyKey` record.

This allows the API to determine whether the same idempotency key is being reused for a different logical request.

For example:

First request:

`Idempotency-Key: abc-123`

`Source Trip: A`

Later:

`Idempotency-Key: abc-123`

`Source Trip: B`

The request hashes differ.

The API returns:

`409 IDEMPOTENCY_KEY_REUSED`

This prevents an idempotency key from being reused to represent multiple logical operations.

17. Stored Response

When the Copy Trip operation succeeds, the resulting response is stored in:

`IdempotencyKey.responseBody`

The copied resource ID is stored in:

IdempotencyKey.resourceId

A retry therefore returns the original result directly.

This has two important properties:

1. No duplicate resource is created.
 2. The retry receives the same logical response as the original request.
-

18. Why try/catch Is Not the Concurrency Mechanism

The `try/catch` block is important, but it is not what prevents the race condition.

The actual concurrency guarantee is:

Database UNIQUE constraint

The role of `try/catch` is to handle the expected result of that constraint.

The flow is:

Database constraint

↓

Prevents duplicate idempotency records

↓

Prisma returns P2002 to the losing transaction

↓

Service catches P2002

↓

Existing completed result is loaded

↓

Original result is returned

Therefore, concurrency control and error handling are separate responsibilities.

19. Automated Testing

Vitest is used for the Copy Trip test suite.

The test file is:

tests/trips/copy.trip.test.ts

The tests cover four important scenarios.

19.1 Concurrent Requests

Two requests are executed concurrently:

```
Promise.all([
  copyTrip(...),
  copyTrip(...),
])
```

The test verifies:

- both calls return the same result
- both IDs are equal
- only one copied trip exists
- only one idempotency record exists
- the idempotency record is completed
- the resource ID points to the copied trip

This directly verifies the concurrency requirement.

19.2 Idempotency Key Reuse

The test first copies Trip A.

It then attempts to use the same key for Trip B.

The expected result is:

```
409  
IDEMPOTENCY_KEY_REUSED
```

This validates request-hash protection.

19.3 Failed Copy Does Not Consume the Key

The test attempts to copy a non-existent trip.

The expected result is:

```
404  
TRIP_NOT_FOUND
```

The test then checks that no IdempotencyKey record exists.

This verifies transactional rollback.

19.4 Completed Request Retry

The same copy request is executed twice sequentially using the same key.

The test verifies:

```
second === first
```

and confirms that only one copied trip exists.

This verifies normal idempotent retry behavior.

20. Seed Strategy

The project provides deterministic seed data.

The seed script:

`src/scripts/seed.ts`

uses fixed UUIDs and Prisma upsert operations.

This makes the seed:

- repeatable
- deterministic
- safe to execute multiple times

The seeded dataset contains:

User

└─ Valizu Case User

Items

└─ Passport

└─ Phone Charger

Trip

└─ Istanbul Trip

└─ Carry-on

└─ Passport × 1

└─ Phone Charger × 1

The seed user is:

`case@valizu.test`

21. Validation

Zod is used to validate incoming request data.

Examples include:

- bag name length
- required request fields
- item UUID
- quantity minimum
- idempotency key length

The API distinguishes validation errors from business errors.

Example:

400 VALIDATION_ERROR

versus:

404 TRIP_NOT_FOUND

This makes the API contract predictable for clients.

22. Error Handling

The API returns structured errors.

Example:

```
{  
  "error": "TRIP_NOT_FOUND",  
  "message": "Trip not found"  
}
```

Validation errors additionally contain:

details

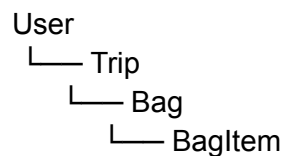
The API documents these error structures in OpenAPI.

This provides consistent client-side error handling.

23. Referential Integrity

The database uses foreign keys and cascading rules.

For example:



Deleting a parent resource can cascade to dependent resources where appropriate.

The BagItem → Item relationship uses restrictive deletion semantics so that an item cannot be deleted while it is still referenced by a bag.

This allows the database to enforce domain integrity independently of application code.

24. Important Database Constraints

The implementation deliberately places important invariants at the database level.

Unique user email

User.email UNIQUE

Unique item inside a bag

BagItem(bagId, itemId) UNIQUE

Unique idempotency operation

IdempotencyKey(userId, operation, key) UNIQUE

These constraints prevent invalid states even if multiple application instances are running concurrently.

25. Why PostgreSQL

PostgreSQL is a good fit for this domain because the data is strongly relational.

The implementation benefits from:

- ACID transactions
- unique constraints
- foreign keys
- referential integrity
- transactional isolation
- reliable concurrency control
- JSON support for stored idempotency responses

The Copy Trip operation in particular benefits from PostgreSQL's transactional guarantees.

26. Why Prisma

Prisma was selected for:

- type-safe database access
- schema-driven development
- migration support
- transaction support
- explicit relational modeling
- structured database errors

The Prisma schema also makes important domain constraints visible in source control.

27. Why TypeScript

TypeScript provides compile-time safety across:

- controllers
- services
- schemas
- database access
- API responses

This reduces the risk of inconsistent data structures and makes the service layer easier to maintain.

28. API Contract as a First-Class Artifact

The OpenAPI document is treated as a first-class part of the project rather than as an afterthought.

It defines:

- endpoint behavior
- request formats
- response formats
- authentication requirements
- idempotency requirements
- validation constraints
- error responses

This makes it possible to use the API specification as a shared contract between backend developers, frontend developers, QA, and API consumers.

29. Design Principles

The implementation follows several core engineering principles.

Single Responsibility

Controllers handle HTTP concerns.

Services handle business logic.

Prisma handles persistence.

Validation schemas handle request validation.

Defense in Depth

Important rules are enforced at multiple layers.

For example:

Application validation

+

Service ownership checks

+

Database constraints

Database as the Final Authority

Concurrency-sensitive invariants are enforced by the database.

This is particularly important for idempotency.

Atomic Operations

Multi-resource operations are executed inside transactions.

Explicit Failure Semantics

Expected failures have explicit application error codes.

Repeatability

The project can be initialized using deterministic seed data.

30. Production Improvements

The case study intentionally uses a lightweight authentication mechanism.

For a production implementation, I would extend the project with:

- JWT/OAuth2 authentication
- RBAC/permission management
- rate limiting
- request correlation IDs
- structured logging
- metrics
- distributed tracing
- security headers
- secret management
- database connection pool tuning
- API versioning
- CI/CD
- automated migration deployment
- database backup and recovery
- audit logging
- API gateway integration

The core Copy Trip idempotency design can remain largely unchanged because it already relies on database-level concurrency guarantees.

31. Conclusion

The implementation does not treat the case study as a simple CRUD exercise.

The main focus is correctness under real-world conditions, particularly:

- concurrent requests
- retries
- duplicate operations
- transaction failures
- ownership boundaries
- database consistency

The Copy Trip implementation demonstrates that idempotency should be designed as a complete system involving:

Request identity

+

Request hash

+

Database uniqueness

+

Transaction

+

Stored response

+

Explicit conflict handling

+

Automated concurrency tests

This approach prevents duplicate copies while preserving correct behavior during retries and failures.

The result is a backend implementation that is intentionally simple at the HTTP layer while maintaining strong consistency guarantees at the service and database layers.

32. Author

Okhtay Sattari

Software Developer & Architect

Backend Case Study